5.2 Webhook与事件驱动

> **生成模型**：OpenAI GPT-5.3 Codex (`openai/gpt-5.3-codex`)
> **Token 消耗**：输入 ~24k tokens，输出 ~4.4k tokens（估算）

Cron 解决“什么时候做”，Webhook 解决“发生了什么就马上做”。把两者放在一起，OpenClaw 就从定时助手变成实时自动化入口。

这一节我们只看实操主线：Webhook 是什么、OpenClaw 怎么接、Gmail Pub/Sub 怎么进、事件驱动有哪些可复用模式。

5.2.1 Webhook 是什么

给没接触过的人一个直观比喻：

- 轮询：你每隔 1 分钟问一次“有新消息吗”
- Webhook：对方有新消息就主动给你打电话

> **衍生解释 - 事件驱动**
>
> “事件驱动”就是有事才触发处理，没有事件就不执行。相比轮询，通常更及时、也更省资源。

常见事件源：GitHub push、表单提交、监控告警、支付状态变化、新邮件到达。

5.2.2 OpenClaw 的 hooks 基础配置

```
{
  hooks: {
    enabled: true,
    token: "OPENCLAW_HOOKS_TOKEN",
    path: "/hooks",
    defaultSessionKey: "hook:ingress",
    allowRequestSessionKey: false,
    allowedSessionKeyPrefixes: ["hook:"],
    allowedAgentIds: ["main", "ops", "hooks"]
  }
}
```

重点字段：

- `token` ： 鉴权令牌，必须启用
- `path` ： 统一 webhook 路径前缀
- `defaultSessionKey` ： 默认会话归类
- `allowRequestSessionKey` ： 建议默认关闭
- `allowedAgentIds` ： 限制外部能唤起的 Agent

Webhook 是外部输入，默认按不可信处理，这些限制是必要配置，不是可选优化。

5.2.3 两个常用 endpoint

`POST /hooks/wake`

适合轻量唤醒某条会话。

```
{
  "text": "CI 构建失败，请检查",
  "mode": "now"
}
```

`POST /hooks/agent`

直接触发一轮 Agent 执行，最通用。

```
{
  "message": "收到 GitHub push，请拉取代码并运行测试，返回摘要",
  "name": "GitHub",
  "agentId": "ops",
  "sessionKey": "hook:github:repo-main",
  "wakeMode": "now",
  "deliver": true,
  "channel": "slack",
  "to": "channel:C0123456789"
}
```

5.2.4 最小可跑示例

配置：

```
{
  hooks: {
    enabled: true,
    token: "OPENCLAW_HOOKS_TOKEN",
    path: "/hooks",
    defaultSessionKey: "hook:ingress",
    allowRequestSessionKey: false,
    allowedAgentIds: ["main"]
  }
}
```

调用：

```
curl -X POST http://127.0.0.1:18789/hooks/agent   -H 'Authorization: Bearer OPENCLAW_HO
    "message": "有新表单提交，请提取姓名、联系方式、问题摘要，输出3条重点。",
    "name": "Form",
    "deliver": true,
    "channel": "telegram",
    "to": "123456789"
  }'
```

这跑通以后，任何支持 HTTP 的系统都能把事件接进 OpenClaw。

5.2.5 mappings：把复杂逻辑留在本地配置

不建议让外部请求携带大量 prompt。更稳的方式是路径映射：外部只负责打某个 URL，内部决定怎么执行。

```
{
  hooks: {
    enabled: true,
    token: "OPENCLAW_HOOKS_TOKEN",
    path: "/hooks",
    mappings: [
      {
        match: { path: "github-deploy" },
        action: "agent",
        wakeMode: "now",
        name: "GitHubDeploy",
        sessionKey: "hook:github:deploy",
        messageTemplate: "收到 GitHub push，请拉取最新代码，运行测试、构建并部署，返回摘要。",
        deliver: true,
        channel: "slack",
        to: "channel:C0123456789"
      }
    ]
  }
}
```

这样外部系统只需 `POST /hooks/github-deploy` 。好处是：外部更简单、配置更集中、安全边界更清晰。

5.2.6 Gmail Pub/Sub 邮件钩子

Gmail 邮件场景是事件驱动自动化的典型：新邮件到达就触发处理。

链路：

```
Gmail -> Google Pub/Sub -> gmail watch 服务 -> OpenClaw /hooks/gmail
```

配置示例：

```
{
  hooks: {
    enabled: true,
    token: "OPENCLAW_HOOKS_TOKEN",
    path: "/hooks",
    presets: ["gmail"],
    gmail: {
      account: "openclaw@gmail.com",
      model: "openai/gpt-5.2-mini",
      thinking: "off"
    }
  }
}
```

初始化命令：

```
openclaw webhooks gmail setup --account openclaw@gmail.com
openclaw webhooks gmail run
```

若你要“新邮件自动摘要并发 Telegram”，可增加 mapping：

```
{
  hooks: {
    enabled: true,
    token: "OPENCLAW_HOOKS_TOKEN",
    presets: ["gmail"],
    mappings: [
      {
        match: { path: "gmail" },
        action: "agent",
        wakeMode: "now",
        name: "Gmail",
        sessionKey: "hook:gmail:{{messages[0].id}}",
        messageTemplate: "收到新邮件。发件人：{{messages[0].from}}；主题：{{messages[0].subj
        deliver: true,
        channel: "telegram",
        to: "123456789"
      }
    ]
  }
}
```

5.2.7 事件驱动模式（实战常用）

- **事件即摘要**：新邮件/工单来了先压缩信息
- **事件即决策**：先判断再执行，不写死一条命令
- **事件即路由**：不同事件发给不同 Agent
- **实时 + 定时兜底**：Webhook 实时处理，再配 Cron 补扫漏单

例如邮件系统可用：实时 Gmail hook + 每晚 `0 23 * * *` 做一次补扫。

5.2.8 安全建议

- 使用独立 `hooks.token`
- 限制 `allowedAgentIds`
- 默认关闭 `allowRequestSessionKey`
- 外网暴露前放到受控代理或内网
- 高风险动作加审批/降权

本节小结

Webhook 让 OpenClaw 从“按时间自动化”扩展到“按事件自动化”。落地顺序建议是：先开基础 hooks 配置，再用 `/hooks/agent` 跑通最小链路，然后用 `mappings` 固化规则，最后接入 Gmail Pub/Sub 这类真实事件源。工程上最稳的组合是“Webhook 实时触发 + Cron 定时兜底”，再配合好 token 和 Agent 白名单，自动化就会既灵活又可控。